Course Syllabus

Edit

Instructor

Name: Dr. David Patrick

Email: ueg11@txstate.edu (mailto:ueg11@txstate.edu)

Include CS 2308 in the subject head as it allows me to respond to these emails faster.

*I typically respond within 24 hours on weekdays.

Office Hours

Hours: MW 12:30 - 2:00 and by appointment

Location: CMAL 210E

If my office door is open (and it isn't within 15 minutes of a class time), feel free to enter.

Teaching Assistants

Name: Kamrul Hasan

Email: sii25@txstate.edu (mailto:sii25@txstate.edu)

Name: Awatif Yasmin

Email: nuc4@txstate.edu (mailto:nuc4@txstate.edu)

*Please allow several hours for a response and do not expect responses over weekends, holidays, or at night.

Course Format

This is a **Face to Face** course.

Section 002 Class Hours: MWF 11:00 am - 11:50 am

Location: Bruce and Gloria Ingram Hall 03104

Textbook

We will be using an online textbook from zyBooks. It is required. Please visit Module 0 on Canvas for how to get access to the Zybook.

Course Description

Fundamentals of object-oriented programming. Introduction to abstract data types (ADTs) including lists, stacks, and queues. Searching and sorting. Pointers and dynamic memory allocation. This course is a continuation of CS 1428.

Course Prerequisites

C or higher in CS 1428: Foundations of Computer Science I

Course Goals and Learning Objectives

At the end of the course, the students should be able to:

- Describe and demonstrate at least two different algorithms for searching and at least two different algorithms for sorting.
- Implement a divide-and-conquer algorithm to solve an appropriate problem (binary search).
- State the time/space efficiency of various algorithms (using one of 6 categories of mathematical functions).
- List the 6 categories of mathematical functions used in analyzing algorithms in order from slowest to fastest growing.
- Read and write C++ code that uses pointer variables and memory operations (new, &, *, delete), including pointers to arrays, structures, and objects and the -> operator.
- Write C++ code that resizes an array using dynamic memory allocation.
- Write C++ code that deletes dynamically allocated memory to avoid memory leaks.

- Describe the basic concepts of object-oriented programming.
- Design, implement, test, and debug simple programs (using objects) in an object-oriented programming language (C++).
- Describe how the class mechanism supports encapsulation and information hiding.
- Develop (implement) programs using multiple classes and arrays of objects.
- Develop and use appropriate algorithms, especially for processing lists (insert, remove, search, sort, etc.).
- Describe structured programming in terms of modules and functions.
- Develop (implement) programs with source code separated into multiple files, including header (.h) files.
- Create, compile, and run a C++ program in a unix style command-line environment.
- Develop (Implement) C++ programs that create and use simple linked-lists, including code to insert into, delete from, and traverse a linked list structure.
- Compare and contrast the costs and benefits of dynamic and static data structure implementations.
- Describe the principle of the Abstract Data Type (ADT) and, in particular, explain the benefits of separation of interface and implementation.
- Implement user-defined data structures in a high-level language.
- Implement the list, stack, and queue ADT using arrays and linked lists.
- Write programs that use each of these data structures: linked lists, stacks, and queues.

Required Material

- ZyBooks

Zybooks

This interactive textbook contains readings, live coding examples from class, and most course assignments. Components typically include Learning Activities, and Programs. Given the substantial number of activities throughout the semester, it is strongly recommended that you pace yourself and complete them consistently rather than attempting to finish everything at the last minute.

Grading

The course is graded out of an 1000 point scale. Below is a breakdown of the course's point

structure:

Course Point Structure

Group	Points
ZyBook Learning Activities	60
Take Home Quizzes (THQs)	80
Programs	260
Exam 1	150
Exam 2	150
Final Exam	300

1. **Rounding Policy:**
    - Final grades are determined based on the total points earned out of 1000. Fractional points will not automatically be rounded up.
    - **Failing Exam Policy:** If a student fails either exam (scores below 70%), they may not qualify for any rounding considerations, regardless of their total points.
    - In cases of borderline scores (e.g., 899 points), adjustments may be made at the instructor's discretion, particularly if the student's overall performance reflects consistent effort and improvement. However, this is not guaranteed.
2. **Grading Scale:**
    - **A:** 900+ points
    - **B:** 800–899 points
    - **C:** 700–799 points
    - **D:** 600–699 points
    - **F:** 0–599 points
3. **Extra Credit Policy:**
    - Extra points may be made available throughout the semester through bonus activities, challenges, or participation incentives. These points can serve as "padding" to help students recover from lost points elsewhere since the grading scale remains fixed at 1000 points.
4. **Flexibility of Point Structure:**
    - The assignment groups and their totals outlined in this syllabus are subject to change. Adjustments may be made to accommodate unforeseen circumstances, to improve the course structure, or to better reflect the workload and learning outcomes. Any changes to the assignment points or grading policy will be communicated through Canvas and during class.

Program Submission and Grading Policy

All submitted programs must compile and run to be eligible for grading. Programs that fail to compile or do not execute correctly will receive a grade of zero. It is your responsibility to thoroughly test your code to ensure it functions as expected. Partial credit will not be awarded for non-functioning programs.

Makeup/Late Policy

Assignments and quizzes cannot be turned in late or made up without permission from the instructor.

Exam Policy

- All tests will be in class during class time.
- There will be no makeup tests. If you have an instructor-approved excuse for missing an exam, your final exam score will be used as the score for both exams. This does not increase the weight of the final; it simply substitutes the missing exam score.

Attendance Policy

Class attendance is highly encouraged but it is not directly included as part of your final grade. Programming is a skills-based activity, and missing class means missing practice. Falling behind is hard to recover from.

Student Disability Accommodations

Any student with needs requiring special accommodations should contact the office of disability services (**ods.txst.edu (http://ods.txst.edu)**). Students who qualify for extra time for exams must take their test with ATSD and must schedule their test at the same time the test is given in

class. You must submit your request with ATSD **at least 3 business days before the exam!**

Helpful Resources

Other than the instructor's office hours, there are other places to obtain assistance:

- **CS Lab tutors** (good for help on MSPs): MCS 593 (inside MCS 590 suite) https://cs.txstate.edu/resources/labs/tutoring/ (https://cs.txstate.edu/resources/labs/tutoring/)
- **CLC tutoring** (good for help on understanding the material): https://hlsamp.cose.txstate.edu/clc.html (https://hlsamp.cose.txstate.edu/clc.html)
- **TA's office hours:** see Canvas for times and how to access.

Academic Honesty

You are expected to adhere to the University's Academic Honor Code: https://www.txstate.edu/honorcodecouncil/Academic-Integrity.html (https://www.txstate.edu/honorcodecouncil/Academic-Integrity.html)

- **You may discuss problems with other students but do not share your code electronically with anyone**. **Do not post your code on group chats or in email!**
- **Do not include code obtained from the internet or any other source in your programs.** Do not use features we have not covered in class without prior approval of the instructor.
- **Do no collaborate with anyone during a test or exam (no talking, no cell phones or smart watches).**
- The penalty for violating academic honesty on an assignment, test or exam is a reduction of points up to a **negative grade** of the assignment, test or exam.

Artificial Intelligence (AI) Use in This Course

What Counts as AI

AI refers to any program or tool that can generate or edit content, including ChatGPT, Copilot, and AI features in development environments.

How You Can Use AI

You may use AI tools to support your learning, such as:

- Brainstorming
- Getting hints or asking clarifying questions
- Generating practice materials
- Reviewing explanations of course material
- Getting help understanding error messages

Think of AI as a study partner. It's helpful for practice and review, but not a replacement for your own work.

What's Not Allowed

- No AI-generated code: All code you turn in must be written and tested by you. Copying, adapting, or "tweaking" AI-generated code counts as prohibited use.
- No AI-submitted work: You cannot submit AI-generated writing or explanations as your own.
- No auto-complete coding from AI: If your IDE/editor has code generation features (e.g., Copilot), you need to turn them off or actively ignore them. You're responsible for ensuring your work is your own.

Why These Restrictions Exist

This course is about developing your own programming and problem-solving skills. If you rely on AI to write your code, you skip the practice you need to become a strong programmer. By writing your own solutions, you'll build skills that AI tools can't give you.

Your Responsibility

If you use AI in permitted ways, you must:

- Make sure you fully understand the explanations it provides
- Write your own original code from scratch
- Be able to explain and reproduce your solutions if asked

Misusing AI not only undermines your learning, but may also violate academic integrity policies. The goal is for you to leave this class confident in your own ability to solve problems and write programs independently.

Withdrawals/Drops

You must follow the withdrawal and drop policy set up by the University.
https://www.registrar.txstate.edu/resources/dropping-vs-withdrawing.html (https://www.registrar.txstate.edu/resources/dropping-vs-withdrawing.html)

Course Schedule

Below is the proposed course schedule:

Course Schedule

Week	Topic
0	Functions, Arrays, & Structs
1	Functions, Arrays, & Structs
2	Classes
3	Classes
4	Pointers & Dynamic Memory
5	Pointers & Dynamic Memory
6	Searching, Sorting, & Analysis
7	Searching, Sorting, & Analysis
8	Midterm Review
9	Linked Lists
10	Linked Lists
11	Stacks & Queues
12	Stacks & Queues
13	Thanksgiving Break Holiday (Optional Lecture)
14	Review

Minor changes to the course schedule may occur throughout the semester to better align with the course content and objectives. All updates and adjustments will be reflected on the live calendar located on the course homepage. It is your responsibility to regularly check the live calendar for the most current information. Any significant changes will also be announced in class and via Canvas announcements.

For a list of important university-wide dates, review **Office of the University Registrar Academic Calendar (https://www.registrar.txst.edu/registration/ac/academic-calendar.html)**

Exam Topics

Exam 1 will cover Units 1, 2, and 3.

Exam 2 will cover Units 4, 5, and 6.

Final exam will be a comprehensive exam of Units 1 - 6.